Case Study 02 — Experimental Only

Measured spectra from PhotochemCAD

Contents

1 Case Study 02 — Experimental Only

Run `overlap-calculator` on **measured / tabulated** absorbance spectra only. No Gaussian TD-DFT files are present in the manifest. The bundled workbook is sliced into two named series so the file count stays small.

Citation. The bundled experimental absorbance data is derived from M. Taniguchi & J. S. Lindsey, *Photochem. Photobiol.* **94** (2018) 290–327. See the project README for the full citation.

1.1 What you will produce

Everything is written under one `output/` directory inside this case study folder:

Folder	By	Contents
`output/cli/tables/`	CLI	Result tables (CSV / JSON / XLSX)
`output/cli/plots/`	CLI	TIFF plots per sample, light source, and (sample, light) pair
`output/api/`	API	Same `tables/` and `plots/` tree, extracted from the ZIP
`output/api/analysis_outputs.zip`	API	Raw ZIP response

Light sources used: **AM1.5G + LED-B4**.

1.2 1. Prerequisites

If you have never run any case study before, follow Case Study 01 §1 Prerequisites once to install the package, the `requests` library, and either the Python or Docker option for the HTTP API. Every case study uses the same environment.

1.3 2. Inputs

The case-study `input.json` lists 2 absorbance series from 1 Excel workbook. The manifest uses relative paths that point at the bundled data under `../../input/files/`, so no files are copied or duplicated. To swap in your own data, replace `input.json` with a manifest of the same shape; you do not need to touch `submit.py`.

1.4 3. Run with the CLI

Open a terminal at the repository root, then run a single command:

```
overlap-calculator analyze --input case_studies/02_experimental_only/input.json --out
↪ case_studies/02_experimental_only/output/cli --default-light-sources AM15G,LEDB4 --plot-dpi 400
```

After the run, `case_studies/02_experimental_only/output/cli/` contains a `tables/` directory (and a `plots/` directory unless plots were disabled). The `--default-light-sources` flag pins the bundled sources to AM1.5G + LED-B4; remove it to compare against the full default set (`AM15G,LEDB4,LEDB2,LEDB3,CIEFL10`).

1.5 4. Run with the HTTP API

Start the API as documented in Case Study 01 §1 and verify with `curl http://localhost:8000/health` (or `:8080` if you started the API via `docker compose`). Then submit the same uploads in one curl command:

```
curl -X POST http://localhost:8000/analyze -F
↪ "files=@input/files/organic_uvvis_photochemcad_dataset.xlsx" -F "default_light_sources=AM15G,LEDB4"
↪ -F "plot_dpi=400" --output case_studies/02_experimental_only/output/api/analysis_outputs.zip
```

Each `-F "files=@..."` is one upload. The form fields mirror the CLI flags. The response is a ZIP archive saved as `output/api/analysis_outputs.zip`.

1.6 5. Drive both interfaces from one Python script

Instead of typing the commands above, run `submit.py` from the repository root:

```
python case_studies/02_experimental_only/submit.py
```

The script reads `input.json`, runs the CLI step (writing to `output/cli/`), then POSTs the same uploads to `/analyze` and unpacks the response into `output/api/`. If the API is not reachable, it prints a hint and exits cleanly without failing the CLI step.

1.7 6. Expected output layout

```
case_studies/02_experimental_only/output/
+-- cli/
|   +-- tables/results.{csv,json,xlsx}        (2 series × 2 light sources = 4 rows)
|   +-- tables/results_timings.{csv,json,xlsx}
|   +-- tables/descriptor_summary.{csv,xlsx}
|   +-- tables/ranking_by_light_source__<metric>.{csv,json,xlsx}   (gaussian/lorentzian x
↪ absorbed_fraction/shape_overlap)
|   +-- tables/ranking_by_sample__<metric>.{csv,json,xlsx}          (same four metric variants)
|   +-- plots/                                                      (also
↪ plots/ranking/by_light_source/<metric>/ and plots/ranking/by_sample/<metric>/)
+-- api/
```

```
+-- analysis_outputs.zip
+-- tables/...
+-- plots/...
```

API sample IDs are auto-generated from each upload's `%chk` header, while the CLI uses the `sample_id` you set in `input.json`. This is the only cosmetic difference between the two trees; the tables align row by row otherwise.

1.8 7. Related case studies

- Case Study 03 — Mixed Theory + Experiment
- Case Study 11 — Sheet Overrides